

Name:- MOHAN MAJI

ROLL:- UG/04/BT CSE/2025/058

- 1) $\Rightarrow$ b) Data and pointer to the next node
- 2) $\Rightarrow$ c) null
- 3) $\Rightarrow$ b) Self-referential structure
- 4) $\Rightarrow$ b) $head == null$
- 5) $\Rightarrow$ b) $newnode \rightarrow next = head; head = newnode;$
- 6) $\Rightarrow$ b) $temp = head; head = head \rightarrow next;$ ^{free temp}
- 7) $\Rightarrow$ c) Traversing from tail to head.
- 8) $\Rightarrow$ b) n

NAME:- MOHAN MAJI

ROLL:- UG/04/BT CSE/2025/058

- 9) $\Rightarrow$ c) Copying the data of $p \rightarrow next$ into, then deleting $p \rightarrow next$
- 10) $\Rightarrow$ b) while ($p \neq null$)
- 11) $\Rightarrow$ b) $p \rightarrow next == null$
- 12) $\Rightarrow$ c) two pointers advancing at diff rate
- 13) $\Rightarrow$ c) Three
- 14) $\Rightarrow$ b) Exactly one node
- 15) $\Rightarrow$ b) $P = (\text{struct node}) \text{ malloc}(\text{sizeof } \text{node})$
- 16) $\Rightarrow$ b) Avoid special-case handling for insertion and deletion at the head
- 17) $\Rightarrow$ c) The first node

NAME:- MOHAN MAJI

ROLL:- UG/04/BT CSE/2025/058

NAME :- MOHAN MAJI

ROLL :- UG/04/BTCSB/2025/058

18) $\Rightarrow$ b) it does not support Random (indexed) access

19) $\Rightarrow$ b) Loss of access to all nodes. Causing to memory leak

20) $\Rightarrow$ b) A Slow pointer and a fast pointer

21) $\Rightarrow$ c) the last node

22) $\Rightarrow$ b) Nodes may be scattered anywhere in memory

23) $\Rightarrow$

```
struct node {  
    int data;  
    struct node * next;  
};
```

24) Initiate list :-

10 $\rightarrow$ 20 $\rightarrow$ 30 $\rightarrow$ 40

Iterative 1 :-

P point to 10,

points to 20.

10 + 20 = 30

Iterative 2 :-

P point to 20;

points to 30

20 + 30 = 50;

NAME :- MOHAN MAJI

ROLL :- UG/04/BTCSB/2025/058

NAME:- MOHAN MAJI
ROLL:- UG/09/BTCSB/2025/058

Iteration 3:-

p points to 30
print 3 30 + 40 = 70

Termination:-

p points to 90
p->next is null
false, so the loop ends.

25) => 1) missing empty list check, head is null
2) head pointer not returned when iterate

Correct Code:-

```
struct node * insertEnd(struct node  
* head, int x) {  
    struct node * t = (struct node *) malloc(  
        sizeof(struct node));  
    t->data = x;  
    t->next = null;  
    if (head == null) {  
        return t;  
    }
```

```
    struct node * p = head  
    while (p->next != null) {  
        p = p->next;  
    }  
    return head;
```

NAME:- MOHAN MAJI
ROLL:- UG/09/BTCSB/2025/058

NAME:- MOHAN MAJI
ROLL:- UG2/04/BTCSB/2025/058

26) $\Rightarrow$

```
node * reverseList (node * head) {  
    node * prev = null, * curr = head;  
    *next = null;  
    while (curr != null) {  
        next = curr -> next;  
        curr -> next = prev;  
        prev = curr;  
        curr = next;  
    }  
    return prev;  
}
```

27) $\Rightarrow$ To delete node curr located B/W
prev and next;

1) Set
prev $\rightarrow$ next = curr $\rightarrow$ next
(bypasses curr).

2) free curr.

Before: [prev] $\rightarrow$ [curr] $\rightarrow$ [next]
after: [prev] $\rightarrow$ [next]

28) Initial: - 5 $\rightarrow$ 8 $\rightarrow$ 12 $\rightarrow$ 3 $\rightarrow$

1) Insert 7 at the beginning

7 $\rightarrow$ 5 $\rightarrow$ 8 $\rightarrow$ 12 $\rightarrow$

2) Delete node containing 12;

7 $\rightarrow$ 5 $\rightarrow$ 8 $\rightarrow$ 3 $\rightarrow$ N

3) Insert 9 at the end:

7 $\rightarrow$ 5 $\rightarrow$ 8 $\rightarrow$ 3 $\rightarrow$ 9 $\rightarrow$ null

NAME:- MOHAN MAJI
ROLL:- UG2/04/BTCSB/2025/058